

Task 7: Query Analysis

I selected a query that retrieves travel packages for a specific destination of Cape Town with the average ratings. I chose this query because it has multiple joins (packages, users, package_destinations, destinations, bookings, reviews) and aggregation. I also generated more data so that I could see a bigger difference with the queries.

The Inefficient Query (See sql file):

I deliberately applied these issues:

- Cross Join before filter- this violated rule 6 and created 54 rows in the Cartesian product.
- Applied a late WHERE clause- this violated rule 2 which filters after joins.
- No indexes on foreign keys- this violated access path optimisation which resulted in full table scans making it slow.

Applying Optimisation (See sql file):

Firstly, I created Indexes to access path optimisation (**See file**). Without indexes, it will do a full table scan, but with indexes it does a binary search.

Secondly, I applied Heuristic Rules:

- Rule 6: Push σ through f - Moved WHERE filter before joins.
- Rule 12: Convert $\sigma + \times$ into f - Replaced CROSS JOIN with INNER JOIN.
- Rule 7: Push π through f - Used subquery for AVG(r.Rating) aggregation.

Performance Comparison:

Before optimisation the query cost estimate was 101.62 and after optimisation it was 1.60 which means that there was a significant 98.4% reduction.

Without indexes the execution time was 0.036296 seconds, and with the indexes it was 0.03555 seconds. This was only a 2.1% improvement because the database still has a relatively small amount of data (and not 10 000+ data to show bigger differences).

Why the huge performance gain?

- Eliminated the Cartesian Product (Rule 12)- Original: 18 packages $\times$ 3 destinations = 54 intermediate rows. Whereas, Optimised: Filter first, then join = 18 rows.
- Did early filtering (rule 6)- The WHERE clause on d.Name = 'Cape Town' is now applied before joins.
- Did index usage- idx_dest_name allows O(1) lookup for 'Cape Town' instead of the long full table scan.

Conclusion:

It is important and recommended to push SELECT operations down the query tree as in Rule 6. You must avoid Cartesian products when joins are appropriate as in Rule 12, and use indexes to enable quick access paths.